

Enhancing Dependability of Machine Learning on COTS GPUs for Space Applications through Fault Masking and Recovery

Dikchhya Kharel

Department of Electrical and
Computer Engineering
Swanson School of Engineering
dik56@pitt.edu

Nischal Kharel

Department of Electrical and
Computer Engineering

Swanson School of Engineering

nik96@pitt.edu

Abstract—Modern space missions are increasingly relying on onboard machine learning to support autonomous decision-making, reduce downlink bandwidth, and enable real-time response to mission events. However, using these algorithms on Commercial Off-The-Shelf (COTS) GPUs is very challenging due to radiation-induced faults such as single-event upsets, transient faults, and long-term ionizing effects. These faults can silently corrupt neural network weights which can lead to incorrect inferences that risk mission safety. This paper investigates lightweight software-based fault masking techniques for GPU-accelerated deep learning, focusing on three strategies: Triple Modular Redundancy (TMR), Dual Modular Redundancy with Retry (DMR+Retry), and Algorithm-Based Fault Tolerance (ABFT). Using controlled fault injections through PyTorchFI, we evaluate the resilience of convolutional neural networks (CNN) under simulated radiation-like soft errors and quantify each technique’s effectiveness in reducing silent data corruption while balancing the tradeoffs between fault coverage and computational overhead. Our results show that TMR provides the strongest fault masking at the cost of significant runtime overhead, while DMR+Retry achieves a significantly more efficient tradeoff between dependability and performance. ABFT offers lightweight error detection with minimal overhead, and when combined with DMR+Retry, it further reduces execution cost while permitting a small increase in silent data corruption compared to TMR or DMR+Retry alone. These findings demonstrate the potential of practical, software-only fault tolerance techniques for enabling dependable machine learning on COTS GPUs in harsh space environments.

Keywords— *dependable computing, fault tolerance, soft errors, COTS GPUs, TMR, DMR+Retry, ABFT, PyTorchFI, space computing*

I. INTRODUCTION

Modern satellites are expected to be more than observers; they must be intelligent enough to analyze data and make decisions in real time. In the past, spacecraft would observe, collect data, and send that raw data back down to the ground station for analysis, but as missions grow more complex and data volumes increase, this approach is becoming less practical. We are moving toward intelligent onboard processing, where spacecraft run machine learning and artificial intelligence algorithms to autonomously analyze data and make decisions without waiting for ground intervention.

The main challenge with onboard processing lies in the hardware. The machine learning and artificial intelligence

needed for onboard processing requires much more computing power than the traditional space qualified processors can provide [10]. Traditional radiation hardened processors are designed to withstand the harsh space environment, but they are limited in performance, expensive, and have long development cycles because they must be thoroughly tested before being certified. As a result, radiation hardened processors often become outdated by the time development is completed. In contrast, commercial off-the-shelf (COTS) processors advance rapidly, incorporate the latest innovations, and offer significantly higher performance at a fraction of the cost, but they do not go through the same extensive radiation qualification and therefore lack radiation tolerance. This tradeoff creates a critical gap. COTS GPUs provide the parallel computing power needed for modern machine learning workloads; however, operating them in a harsh environment like space makes them very vulnerable to radiation induced faults such as single event upsets, single event transients, single event latch up, and total ionizing dose effects. These faults are particularly concerning for neural networks because even a single bit flip in a weight can alter feature maps, causing errors that may cascade through the network and degrade the model’s final prediction.

Space missions increasingly rely on onboard machine learning for autonomy, so they cannot accept a high failure rate from COTS hardware or fall back entirely to traditional radiation hardened systems that sacrifice computational capability. Hardware based solutions such as radiation hardened GPUs and triple modular redundancy can mask faults, but they are often too costly and power hungry for small satellites and resource constrained missions. This motivates the need for software and information level fault tolerance that can be deployed on existing COTS GPUs to provide resilience while preserving performance and cost.

In this context, software-based fault masking techniques for GPU accelerated machine learning are an attractive approach to improving dependability without requiring new hardware. By evaluating and comparing such techniques under realistic soft error conditions, we can better understand how to reduce silent data corruption in deep neural networks on COTS GPUs and support more dependable autonomous space missions in harsh radiation environments.

II. BACKGROUND AND RELATED RESEARCH

A. Radiation Effects on COTS Computing Platforms

Every space mission includes at least some electronics that are not designed with space application as a primary use case. These electronics are nonetheless deployed in space missions, where they are exposed to a much harsher environment than they were originally designed for. Any component being used in space missions will have to deal with some level of radiation originating from galactic cosmic rays, trapped radiation belts, solar or deep space events. When radiation particles interact with semiconductor devices, they can deposit charge and disrupt normal circuit behaviors.

These interactions give rise to radiation-induced faults including single-event upsets (SEUs), which flip stored bits; single-event transients (SETs), which introduces short-lived voltage spikes; single-event latchups (SELs), which causes transistors to get stuck at unintended state and cumulative degradation due to total ionizing dose (TID). While latchups and TID effects can cause system reset or permanently damage components, SEUs and SETs are particularly challenging because they may not immediately result in failures.

In a mission-critical systems, silent data corruption (SDC) caused by transient radiation effects presents a serious risk. Computations may be completed successfully while producing incorrect outputs, making such errors difficult to detect and correct without explicit fault tolerance mechanisms. As space missions increasingly incorporate onboard autonomy and data driven decision making, the tolerance for undetected computational errors continues to decrease. This has motivated growing interest in developing more dependable computing techniques that mitigate the radiation effects without relying exclusively on hardware-based prevention.

B. Vulnerability of GPUs and Neural Network Workloads

This challenge becomes sharper when missions use GPUs to run neural networks for onboard perception, prioritization, or autonomy. GPU neural network workloads are vulnerable to transient faults because many errors do not stop execution. A fault can perturb an intermediate value and still produce an output that looks plausible, so the failure mode often becomes silent corruption rather than a crash [1]. System-level radiation testing on an NVIDIA Xavier SoC also reinforces this point: faults can occur even when the observed application output does not visibly fail, which means correctness checks based only on “did it crash” or “did the output look normal” can miss events [2].

The hardware side of the story is that GPUs and accelerator-like designs depend heavily on fast local storage and reuse. Neural inference repeatedly reuses weights and activations, so if a fault lands in a frequently accessed buffer or cache-related structure, that corrupted value can be consumed multiple times and its influence can spread widely through later computations [1]. In the Xavier proton experiments, the most sensitive behavior they observed was tied to CPU-side cache structures, highlighting how memory hierarchy effects and cache behavior can dominate observed vulnerability even in GPU-capable platforms [2].

The workload side is equally uneven. Neural networks are not uniformly sensitive across layers or data values: some operations can reduce the impact of a disturbed value, while other stages allow errors to pass through more directly into the final prediction [1]. At the same time, whether errors are actually seen at the application level depends on how the workload is mapped and monitored across the system. Rodríguez-Ferrández et al. show that under their test conditions, GPU-side errors did not surface as output corruption, while CPU-side structures showed clear sensitivity, so the vulnerable “weak link” can shift depending on where computation and control are happening [2]. Looking at them together, these results indicate that GPU neural workload vulnerability is a system-level phenomenon driven by both architectural behavior and how errors propagate through the model’s computation [1], [2].

C. Fault Injection as a Tool for Reliability Evaluation

Given that GPU neural workloads can fail silently, we need a way to study errors that is repeatable, scalable, and still representative of how faults propagate through a real software stack. Fault injection fits that role by intentionally introducing controlled corruptions during execution and then classifying what happens, for example whether the run masks the fault, crashes, or completes with an incorrect output. Both NVBitFI and SASSIFI frame fault injection as a practical technique for measuring end-to-end error effects and for generating vulnerability data that you can actually use to guide mitigation decisions [3], [4].

NVBitFI is aimed at making GPU fault injection usable on modern workloads without needing source-level changes. The core idea is dynamic and selective binary instrumentation of GPU kernels, so injections can be applied to the minimal set of target kernels as they execute. This improves practicality for real applications that use dynamic libraries or where source code is not available, and it helps reduce overhead compared to less selective approaches [3].

SASSIFI targets the GPU at the architecture-visible level by injecting faults into low-level state through SASS (GPU assembly) instrumentation using SASSI. It supports injecting into different kinds of architectural state such as registers and memory, and it provides an automated campaign workflow that profiles the execution, selects injection sites statistically, and runs injections while monitoring outcomes like crashes, hangs, and output corruption. This makes it useful for studies where you want to connect low-level corruptions to application-level SDC rates and identify which instruction types or execution regions are most sensitive [4].

A complementary approach is framework-level fault injection which introduces perturbations directly into the neural network computation at runtime. PyTorchFI is a PyTorch-based perturbation tool that injects controlled faults into model parameters or intermediate values such as neuron and activation tensors during inference, with injection sites specified using model-level constructs like layers and tensor indices rather than GPU instructions or microarchitectural state [9]. This makes it

well suited for experiments that evaluate software fault-tolerance mechanisms in the same execution context used for deployment, since techniques such as redundancy, retry, or ABFT ultimately aim to reduce output-level silent data corruption and maintain task accuracy under corrupted internal values [9]. While this abstraction does not explicitly model which hardware structure experienced an upset, it provides a repeatable way to quantify how sensitive a given model and operator implementation is to perturbations at specific locations, and to measure how much a software mitigation shifts the rates of incorrect outputs versus detected mismatches or recoverable failures [9].

D. Software-Based Fault Tolerance for Neural Networks

Software-based fault tolerance for neural network inference focuses on improving correctness on COTS GPUs using methods implemented in software, rather than relying on radiation-hardened hardware. In this context, software-based means the mitigation lives in the execution flow, the operator implementation, or the model graph, so it can be deployed on existing GPU platforms and adjusted per workload. Recent work spans redundant execution and structured checking, with the practical goal of reducing silent data corruption while keeping overhead acceptable for real deployments [6], [5], [7].

Redundancy-based approaches provide a clear baseline for masking and detection, but the GPU execution model makes implementation choices consequential. Software-only triple diverse redundancy proposes TMR at kernel granularity by triplicating inputs, outputs, memory operations, and kernel launches, while placing redundant kernels into different CUDA streams to encourage concurrent execution and allocation across different SMs; results are then majority-voted on the CPU/MCU side [6]. The same work highlights that software diversity does not eliminate all common-cause failure modes, particularly when non-redundant GPU components such as scheduling infrastructure are shared, and it treats this limitation as a motivation for careful evaluation rather than as a solved problem [6]. The evaluation is positioned around timing impact instead of fault injection, reporting that execution time is generally below a 3x increase relative to the non-redundant baseline and that output comparison costs are small for most kernels [6]. This body of work is directly relevant to TMR and also motivates DMR+Retry as a lower-overhead point in the same design space, where mismatches can trigger recovery actions rather than requiring permanent triplication.

Algorithm-Based Fault Tolerance (ABFT) targets lower overhead by exploiting mathematical structure in dominant operators. FT-CNN proposes checksum-based ABFT schemes for convolution and integrates them into a workflow designed to provide detection and correction while limiting runtime overhead; evaluation across common CNN models reports low overhead in both error-free and injected-error settings [5]. Arithmetic-intensity-guided fault tolerance extends this direction for GPU inference by observing that neural networks contain a mix of compute-bound and bandwidth-bound layers, then selecting among ABFT schemes per layer using arithmetic

intensity so that protection aligns with each layer’s bottleneck; reported results show reduced overhead compared to using a single ABFT scheme throughout the network [7]. These ABFT approaches connect cleanly to DMR+Retry style recovery logic: when detection is prioritized, a detected event can trigger replay or rollback at an appropriate scope, while correction is reserved for cases where it can be applied efficiently.

In addition to redundancy and ABFT, some software techniques aim to reduce fault impact without explicitly replicating computation or adding checksum structures to every operator. Ranger follows this direction by leveraging the empirical observation that many intermediate activations and layer outputs remain within bounded ranges during fault-free inference, while transient faults can introduce large out-of-range excursions that are more likely to alter final predictions [8]. Its approach profiles these value ranges offline and then applies lightweight online truncation at selected layers to constrain extreme deviations at runtime [8]. The reported evaluation shows large reductions in silent data corruption with negligible overhead and without degrading nominal accuracy under the tested bounds, making range restriction a useful example of a low-cost complement to stronger but heavier-weight mechanisms such as TMR or ABFT [8]. Ranger is not a substitute for redundancy or ABFT, but it represents a different lever: instead of replicating computation or adding checksums, it constrains the magnitude of fault manifestations so the network’s inherent tolerance can absorb remaining small deviations [8]. Together, redundancy (TMR and DMR+Retry), ABFT, and range restriction span a practical spectrum of software-only options for improving neural inference dependability on COTS GPUs.

III. APPROACH

A. Approach Overview

This work evaluates how transient, radiation-like single event upsets affect convolutional neural network inference and how software-level fault tolerance techniques influence both correctness and execution cost. Our experiments use AlexNet in inference mode with pretrained ImageNet weights, and input images are randomly selected from the DOTA-v1.5 dataset, which provides realistic, high resolution aerial imagery and visual complexity that is representative of remote sensing workloads. Given that AlexNet is not trained on DOTA classes and our goal is reliability rather than task performance, we do not evaluate classification accuracy against dataset labels. Instead, we quantify resilience by comparing the output produced under fault injection against fault-free golden output for the same input and determining whether they match.

Faults are injected during the forward pass using PyTorchFI, and we evaluate a baseline configuration along with Triple Modular Redundancy (TMR), Dual Modular Redundancy with Retry (DMR+Retry), a checksum-based ABFT detector, and a combined ABFT + DMR+Retry configuration. Across repeated trials, we record silent data corruptions (SDCs), detected unrecoverable errors (DUEs), retry frequency, and runtime overhead to characterize the tradeoffs between the different techniques in terms of fault coverage and performance.

B. Project Evolution and Design Decisions

This work initially targeted architecture-level fault injection using NVBitFI [3] on an NVIDIA Jetson Orin Nano platform. NVBitFI provides low-level fault injection by modifying register values during kernel execution offering realistic emulation of radiation-induced bit flips in hardware state.

During Implementation, we encountered a compatibility issue with NVBitFI's injector component on SM_87 (Ampere) architecture. While NVBit 1.7.6 officially supports Ampere GPUs for instrumentation and profiling operations, the specific runtime device code linking required by NVBitFI's register-level injection mechanism has not been validated for this architecture. NVBitFI's profiler component executed successfully on our platform, confirming the limitation was specific to the injection mechanism rather than general instrumentation support.

We therefore adopted PyTorchFI [9], a framework-level injection tool, to directly inject faults into the activation tensor during forward pass of neural network models. While this pivot does sacrifice hardware-level injections, it enables repeatable, controlled fault campaigns that allow for direct evaluation of software mitigation effectiveness at the abstraction level where these techniques operate.

We selected AlexNet as our evaluation model due to its sequential convolutional structure, which provides clearer fault propagation patterns compared to more complex architecture with high internal redundancy like YOLO models. This makes it a perfect choice for a controlled comparative study of fault tolerance.

C. Hardware and Software Environment

All experiments were run on an NVIDIA Jetson Orin Nano platform using GPU-accelerated PyTorch inference. The fault injection pipeline was implemented in Python using PyTorch and torchvision, and injections were performed with PyTorchFI during the forward pass. The same platform configuration, model weights, and preprocessing were used across techniques to ensure a fair comparison of fault tolerance behavior and runtime overhead.

D. Model and Input Data

The neural network model used for the study was AlexNet in inference mode with pretrained ImageNet weights. The model was executed using the standard torchvision AlexNet configuration and was set to evaluation mode so that training-time behaviors did not influence outputs. Each trial performed a forward pass on a single input image and produced an output prediction that was then used for fault tolerance decisions and for comparison against a fault-free or golden run output.

Input images were selected at random from the DOTA-v1.5 dataset consisting of realistic remote imagery. Since AlexNet was not trained in DOTA classes and DOTA labels, classification accuracy was not collected. Instead, correctness was determined relative to the model's fault-free output for each technique given the same image. A trial was considered corrupted when the accepted output under fault injection

differed from the corresponding golden run output of the same image.

Each input image was preprocessed using the standard ImageNet-style pipeline required by AlexNet, including resizing the images to the expected size and normalizing to stay consistent with the pretrained weights. All preprocessing steps stayed identical across all the techniques to ensure that any difference recorded due to fault injection is because of fault tolerance techniques.

E. Fault Injection Methodology

Fault injection was performed at the framework level during the model forward pass to emulate a transient radiation-like single event upset. Each injection was for a single-bit flip applied to one intermediate activation value in the output feature map of convolution layer. This models transient corruption during computation without having to modify the persistent model weights.

Each trial begins by selecting one random image from the dataset, then a fault-free reference for the same image is loaded from a precomputed golden run. Golden run outputs are generated once for each technique by running the same model on each validation image without fault injection and saved as golden outputs for comparison.

For each technique, injection is implemented using a forward hook on a randomly selected AlexNet convolution layer. Each inference execution had a 50% probability of receiving one injected fault. This probabilistic injection mimics the intermittent nature of transient radiation effects, where many executions complete fault-free and occasional executions experience a single upset. It also interleaves clean and faulty trials in the same campaign, so fault-free runs serve as an internal control for measuring baseline variability and false positives under identical conditions. When injection is enabled, one convolution layer is randomly chosen from the model's convolution layers. During the layer's forward pass, the layer output tensor is cloned, a single element is selected at a random channel and spatial location and a single random bit in that activation value is then flipped. The modified activation output is then returned so the remainder of the network propagates the fault naturally through subsequent layers.

For techniques that execute the model multiple times per trial, injection decisions are made for each execution separately and independently. For example, in TMR, three executions are performed and each execution independently may receive an injected fault or be fault-free. This is to simulate the transient behavior of radiation-induced faults, they can exist for some runs and be absent for others. In ABFT with DMR+Retry, additional executions are only performed when ABFT detection triggers, and each triggered execution follows the same injection pipeline.

F. Fault Tolerance Techniques Implemented

This study evaluated four protection configurations in addition to one unprotected baseline. Across all configurations, the same model, inputs, and preprocessing were used to keep

all technique inferences consistent for comparison. Differences between configurations arise only from the fault tolerance logic applied to one or more executions.

1) *Baseline (no technique)*: A single fault-injected inference executed.

2) *Triple Modular Redundancy (TMR)*: Three independent inferences were executed and the final output was selected by majority vote across the three outputs; if no majority was possible, the run was counted as a DUE.

3) *Dual Modular Redundancy with Retry (DMR+Retry)*: Two inferences were executed and compared; if they matched, the output was accepted, and if they disagreed, a third retry inference was executed and majority voting was used to determine the final output.

4) *Algorithm-Based Fault Tolerance (ABFT) detection*: For each convolution layer, we computed a checksum output in two ways: (1) by summing the layer's output channels, and (2) by applying a checksum filter derived from the layer weights to the same input. If the difference between these two checksum outputs exceeded a threshold, the run was flagged as potentially corrupted.

5) *ABFT with DMR+Retry*: We used ABFT as a trigger: if ABFT did not flag the initial run, the output was accepted; if ABFT flagged corruption, we ran DMR+Retry (two runs plus a third on disagreement) and used comparison and majority voting to decide the final output.

G. Metrics Collected

To compare fault tolerance techniques under the same injection conditions, we collected metrics that capture both reliability and execution cost. All correctness metrics were computed by comparing the final output of fault injected run with a fault-free golden run for the same input image.

1) *Silent Data Corruption (SDC)*: The final output differs from the golden output and the technique does not flag an error.

2) *Detected Errors*: The technique indicates potential corruption in the output.

3) *Detected Unrecoverable Error (DUE)*: The technique cannot produce a final decision, such as when three executions produce three different outputs and no majority vote is possible.

4) *Average Runtime*: Average wall-clock time per trial to produce the final output, including any retries. Overhead is computed relative to the baseline runtime.

5) *Retry Frequency*: Fraction of trials that required additional executions beyond the first run.

IV. EXPERIMENTS

A. Experimental Design and Trial Procedure

Each experiment was organized as repeated, 10,000 independent trials using a fixed model and fault injection setup. In each trial, one input image was selected at random from the available DOTA-v1.5 validation image set. A fault-free golden output for the same image was loaded from a precomputed golden run. The selected technique was then executed on the image and produced either a final accepted output or a DUE. The final accepted output was compared against the golden output for the same image to determine whether silent data corruption occurred. For every trial, we also recorded total runtime and any technique-specific events such as detection triggers or re-tries. This trial pipeline was identical across configurations to ensure that differences in outcomes were due to the fault tolerance techniques rather than changes in the input or execution pipeline and was repeated until 10,000 trials were collected for each technique or configuration.

B. Experiment Configurations

We evaluated the following configurations:

- 1) *Baseline (no technique)*
- 2) *TMR*
- 3) *DMR+Retry*
- 4) *ABFT detection-only*
- 5) *ABFT with DMR+Retry (ABFT-triggered redundancy)*

In addition, ABFT-based methods were evaluated using a range of thresholds, described in Section IV-D, to observe how detector sensitivity changes behavior and overhead. Threshold tested were:

$$\text{Threshold} \in \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 1.0\}$$

Each configuration was executed for 10,000 trials under the same platform and injection settings.

C. Fault Injection Campaign Parameters

All configurations used the same framework-level fault model. Faults were injected during the forward pass using PyTorchFI to emulate transient, radiation-like single-event upsets. Each injected fault was a single-bit flip applied to one of the intermediate activation values in the output feature map of a randomly selected convolution layer. When injection was enabled for an execution, a convolution layer was selected at random from the AlexNet convolution layers, and a single output element was selected at a random channel and spatial location. At most one fault was injected per execution.

Each injection was enabled independently for every execution with probability $p = 0.5$. For techniques that required multiple execution per trial, each execution included an independent fault injection decision. This means a trial could

contain a mixture of fault-free and fault-injected executions simulating a realistic transient SEU.

D. ABFT Threshold Design

ABFT detection is controlled by a threshold parameter that determines when a convolution layer is flagged as potentially corrupted. The core idea is that for a given convolution layer, we can compute a checksum value for the layer output in two different but equivalent ways. If the layer output is correct, these two checksum values should be close. If a fault corrupts the layer output, they can have a big difference between them. For each protected convolution layer, ABFT computes 2 checksum signals:

- 1) Output checksum from the produced feature map: after computing the layer output feature map Y , ABFT forms a checksum map by summing Y across the output channels. This produces a single 2D checksum image that represents the channel-summed output of the layer.

- 2) Expected checksum using a checksum filter derived from weights: ABFT constructs an additional checksum filter W_{checksum} directly from the layer's convolution weights W . Specifically, W_{checksum} is formed by summing the weights across the output-channel dimension, so that the checksum filter represents the combined effect of all output filters. The checksum output is then computed by applying W_{checksum} to the same layer input X (and similarly combining the bias terms). This produces an expected checksum map with the same shape as the channel-summed output checksum.

The detector then computes the maximum absolute difference between the two checksum outputs. If this maximum difference is greater than the threshold, then this run is marked as having a fault. Smaller threshold values make the detector more sensitive; however, it will increase the number of false positives as well. Larger threshold values are more conservative and reduce false positives but may allow smaller corruptions to go undetected.

E. Runtime Measurement Method

Runtime was measured as the time required for the technique to produce a final decision for a given input, focusing only on model execution and the decision logic needed to accept an output. Timing excluded one-time setup costs and non-compute overhead such as image loading, preprocessing, printing, and logging. For techniques that executed the model multiple times within a trial (TMR, DMR+Retry, and ABFT-triggered DMR+Retry), the recorded runtime included the total time of all model executions performed for that trial, including any additional retry runs. Average runtime was reported as the mean per-trial runtime across the 10,000 trials for each configuration, and runtime overhead was computed relative to the baseline average runtime.

V. RESULTS AND ANALYSIS

A. Baseline fault sensitivity (no protection)

We first measured how often a single transient activation fault leads to an incorrect final model decision when no protection is applied. Across 10,000 baseline trials, fault injections occurred in 4,999 trials and the remaining 5,001 trials were fault-free. Under this setup, the unprotected baseline produced 86 silent data corruptions (SDCs), corresponding to an SDC rate of 0.86% per trial and average inference time of 14.15 millisecond. This indicates a natural masking rate of 98.3% for the injected faults. This high inherent fault masking in neural networks likely stems from their redundant representations in each layer.

However, while natural masking provides significant protection, an 0.86% SDC rate remains unacceptable for critical space applications where reliability requirements are very strict depending on the severity of the mission. This motivates the need for explicit fault tolerance techniques to achieve the necessary reliability standards.

B. Triple Modular Redundancy (TMR)

TMR achieved a complete SDC elimination (0.00% SDC rate) by running three independent inferences and voting on the results. Out of the 10,000 runs, 97.6% achieved three-way agreement, while 2.4% required voting between two matching results. Only 5 runs resulted in Detected Unrecoverable Errors (DUE) where all three runs produced different outputs. The low DUE rate (0.05%) confirms that it is incredibly rare for all three runs to generate conflicting results, and almost impossible for 2 runs to generate the same faulty result, verified by 0% SDC. This demonstrates the high reliability provided by the TMR technique.

However, due to TMR needing 3 separate runs for the voting mechanism, the average inference time is 42.24 milliseconds representing a $2.99\times$ runtime overhead compared to baseline. This added overhead might not be ideal for time sensitive missions.

C. Dual Modular Redundancy with Retry (DMR+Retry)

DMR with voting achieved perfect reliability (0.00% SDC rate, 0 DUEs) while reducing overhead to $1.98\times$ (28.08 ms average time). DMR executed two independent inferences, and when they agreed (98.5% of the cases), immediately accepted the result. When the two runs disagreed (1.5% of the cases), a third run is executed as a tiebreaker. This creates a three-way majority voter similar to TMR.

The adaptive ability of this feature is what provides the reliability of TMR while operating more efficiently and achieving much lower time overhead. This makes DMR+Retry an optimal balance for scenarios where perfect reliability is required, but computational resources are also constrained.

D. ABFT Detection: Threshold Sensitivity Analysis

Algorithm-Based Fault Tolerance (ABFT) using checksum verification shows a critical tradeoff between detection sensitivity and false positive rate, controlled by the detection threshold parameter. Figure 1 illustrates this relationship across threshold values from 0.0 to 1.0.

At a very low threshold (0.0 or 0.1), ABFT achieves near perfect detection rate but suffers significantly from the high false positive rate. This occurs because floating-point arithmetic introduces small noise across the calculations in the model inference and when the threshold is so strict, all runs are flagged as erroneous. Due to such high false positive rates, unnecessary re-execution can be caused, negating any efficiency benefits.

As threshold increases to .2 or .3, the detection rate stays relatively high while false positives continue to decrease. However, the false positive rates still remain above what would be practical.

At a threshold of .5, ABFT reaches a practical balance point, where detection rate remains relatively stable around 28.7% and false positives of only around 1.7%. This detection rate might seem low, but we must compare it with baseline data context. Even when 4986 runs had faults injected, 1430 were detected with only 84 false positives and 3533 were masked. Only 23 faults resulted in undetected SDCs - a 73.3% reduction from the baseline's 86 SDCs.

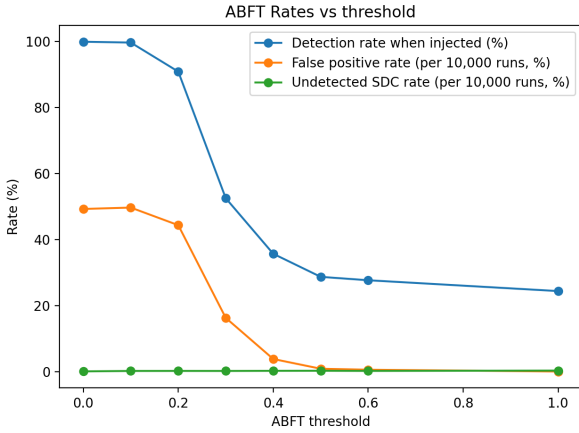


Figure 1: Effect of ABFT threshold on detection accuracy and false positive rate

Higher thresholds do not seem to provide any more protection, allowing a threshold of 0.5 to be an optimal operating point where false positives are minimized while maximizing the detection rate. Making it suitable for cases where moderate reliability improvement is acceptable.

E. ABFT with Dual+Retry: Combining Detection with Correction

Combining ABFT detection (threshold 0.5) with dual execution and retry creates a hybrid technique that attempts to optimize both reliability and performance. When ABFT detects a potential fault, the system performs a second inference and compares results and if they match, the result is accepted; if

they differ, a third inference is executed to allow for majority voting.

At the threshold of 0.5, this technique results in ABFT re-execution being triggered in 1,593 cases (15.93%) where 1513 (95.0%) of those are resolved by agreement between the first two runs. Only 1 case resulted in DUE from triple disagreement.

However, the hybrid approach does result in SDC rate of 0.24% (24 undetected SDCs), which is higher than 0.0% of DMR+Retry alone. This loss in reliability is slightly covered by the decrease in overhead to only 22.43 milliseconds which is 20% reduction in time per image in DMR+Retry.

Across different thresholds (Figure 1), the hybrid approach shows similar patterns to detection-only ABFT, with threshold 0.5 again providing the best balance. However, Figure 2 demonstrates that the hybrid approach does not significantly outperform detection-only ABFT in reducing undetected SDCs across any threshold value, while consistently incurring higher runtime costs (Figure 3).

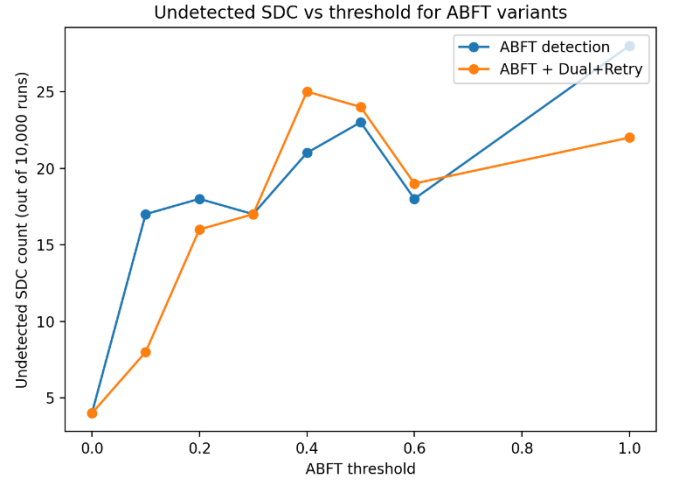


Figure 2: ABFT detection (blue) and ABFT with Dual+Retry (orange) achieve similar undetected SDC counts across all thresholds

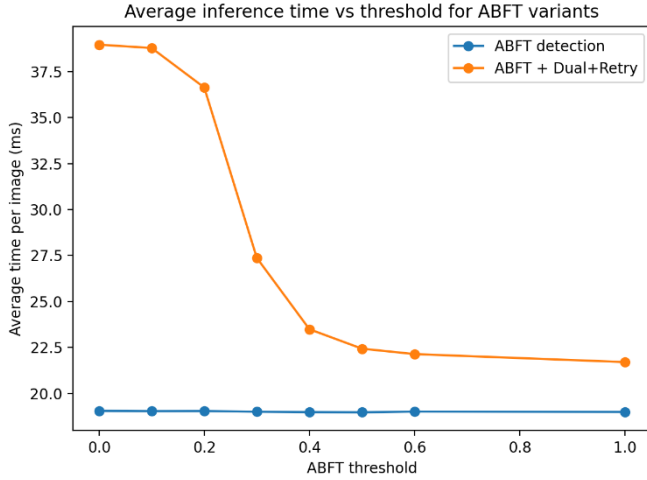


Figure 3: Detection-only ABFT maintains constant overhead while hybrid approach overhead decreases with threshold due to reduced re-execution frequency

F. Comparative Analysis

Technique	Avg runtime (ms)	Runtime overhead (x baseline)	SDC count (%)
Baseline	14.15	1.00	0.86
ABFT (thr=0.5)	18.97	1.34	0.23
ABFT + DMR (thr=0.5)	22.43	1.59	0.24
Dual+Retry	28.08	1.98	0.00
TMR	42.24	2.99	0.00

Table 1: summarizes the performance-reliability tradeoffs across all evaluated techniques

Three distinct design points come from the analysis of Table 1:

1) Perfect Reliability: Both TMR and Dual+Retry resulted in 0% SDCs, representing the highest standard of reliability. Between the two, DMR+retry offers 34% efficiency improvement over TMR (1.98x vs 2.99x overhead), making DMR+retry a preferred choice for fault coverage.

2) Moderate Protection with Low Overhead: ABFT detection (threshold 0.5) reduces SDCs by 73.3% with only 34% overhead, offering a compelling middle ground between the reliability and performance.

3) Hybrid Approach: The ABFT-triggered DMR+Retry combination is attractive due to it being able to both detect and correct faults. The overhead increases from 1.59x to 1.34x but provides virtually no additional SDC reduction.

These tradeoffs are visualized in Figures 4 and 5, which show the progressive increase in runtime overhead

(14→19→22→28→42 ms) alongside the corresponding improvement in reliability (0.86%→0.23%→0.00% SDC rates).

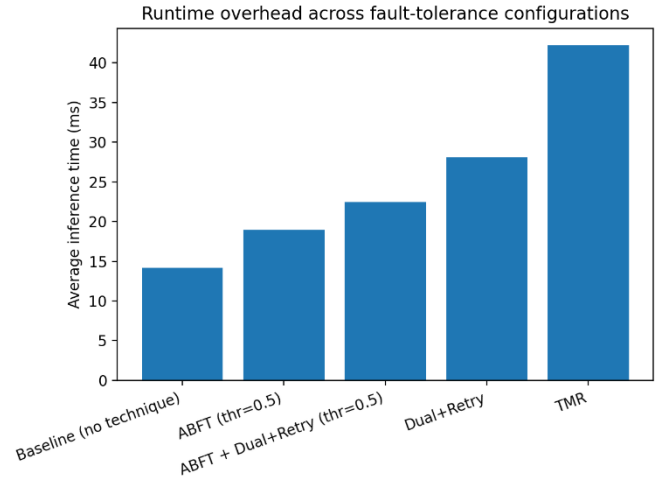


Figure 4: Average inference time comparison across techniques

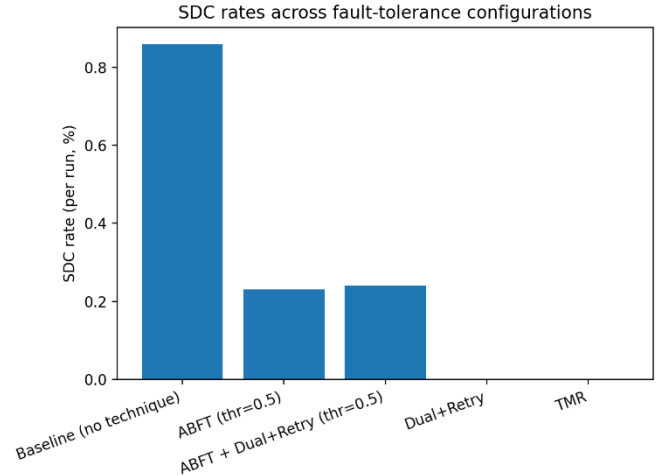


Figure 5: SDC rate comparison across techniques

VI. CONCLUSIONS

This work evaluated five fault tolerance techniques for neural network inference under radiation-induced faults. The results establish clear reliability-efficiency tradeoffs for space AI systems.

Dual Modular Redundancy with retry achieved perfect reliability with 1.98x overhead compared to the baseline with no technique and 34% more efficient than TMR. ABFT detection at a threshold of 0.5 reduced SDCs by 73% with only 1.34x overhead, suitable for applications tolerating a moderate reliability. The ABFT with DMR+Retry hybrid approach showed no SDC improvement over ABFT-only despite 18% higher time, revealing that detection and correction mechanism do not work effectively together.

Mission requirements should drive technique selection: safety-critical applications require DMR+Retry's guaranteed protection; science missions can use ABFT's

minimal overhead, and power-constrained platforms must rely on ABFT as the only viable option.

Future work should explore layer-selective protection, and adaptive thresholding. As neural network applications grow in space systems, this work provides a quantitative baseline to balance reliability, power and performance for AI-enabled space missions.

VII. ACKNOWLEDGEMENTS

We thank the course TAs for their guidance during the project development phase. Their technical insights regarding NVBitFI compatibility limitations on SM_87 architecture helped us identify PyTorchFI as a viable alternative framework-level injection approach, enabling us to proceed with controlled fault tolerance evaluation despite platform constraints.

VIII. REFERENCES

- [1] G. Li et al., "Understanding Error Propagation in Deep Learning Neural Network (DNN) Accelerators and Applications," SC17: International Conference for High Performance Computing, Networking, Storage and Analysis, Denver, CO, USA, 2017, pp. 1-12
- [2] I. Rodriguez-Ferrandez, M. Tali, L. Kosmidis, M. Rovituso and D. Steenari, "Sources of Single Event Effects in the NVIDIA Xavier SoC Family under Proton Irradiation," 2022 IEEE 28th International Symposium on On-Line Testing and Robust System Design (IOLTS), Torino, Italy, 2022, pp. 1-7
- [3] T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa and S. W. Keckler, "NVBitFI: Dynamic Fault Injection for GPUs," 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Taipei, Taiwan, 2021, pp. 284-291
- [4] S. K. S. Hari, T. Tsai, M. Stephenson, S. W. Keckler and J. Emer, "SASSIFI: An architecture-level fault injection tool for GPU application resilience evaluation," 2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Santa Rosa, CA, USA, 2017, pp. 249-258
- [5] K. Zhao et al., "FT-CNN: Algorithm-Based Fault Tolerance for Convolutional Neural Networks," in IEEE Transactions on Parallel and Distributed Systems, vol. 32, no. 7, pp. 1677-1689, 1 July 2021
- [6] S. Alcaide Portet, L. Kosmidis, C. Hernandez and J. Abella, "Software-Only Triple Diverse Redundancy on GPUs for Autonomous Driving Platforms," 2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S), Valencia, Spain, 2020, pp. 82-88
- [7] J. Kosaian and K. Rashmi, "Arithmetic-intensity-guided fault tolerance for neural network inference on GPUs". In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2021, pp. 1-15
- [8] Z. Chen, G. Li and K. Pattabiraman, "A Low-cost Fault Corrector for Deep Neural Networks through Range Restriction," in IEEE Design & Test
- [9] A. Mahmoud et al., "PyTorchFI: A Runtime Perturbation Tool for DNNs," 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), Valencia, Spain, 2020, pp. 25-31
- [10] B. Zhang et al., "Progress and Challenges in Intelligent Remote Sensing Satellite Systems," IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 15, pp. 1814-1822, 2022